

PABD — Programação e Administração de Banco de Dados

Construindo APIs com FastAPI

Uma abordagem alternativa ao Django REST Framework e ao Node.js/Express: endpoints stateless, CRUD de veículos e persistência com PostgreSQL via SQLAlchemy

Sumário

1. Introdução e contexto pedagógico
2. Por que considerar o FastAPI?
3. Configuração inicial do ambiente
4. Primeiro endpoint: "Hello, API"
5. Modelando o recurso Veículo
6. CRUD stateless: GET, POST, PUT/PATCH e DELETE
7. Documentação automática e testes manuais
8. Persistência com PostgreSQL e SQLAlchemy
9. Refatorando o CRUD para usar o banco de dados
10. Exercícios propostos e próximos passos

1. Introdução e contexto pedagógico

Esta aula foi preparado para a disciplina de Programação e Administração de Banco de Dados (PABD), do 4º semestre do curso de Tecnologia em Análise e Desenvolvimento de Sistemas (TADS), como material de apoio para a introdução do framework FastAPI. Até este ponto da disciplina, os(as) estudantes já tiveram contato com a construção de APIs REST utilizando Django REST Framework e Node.js com Express. O objetivo aqui não é substituir esse conhecimento, mas ampliar o repertório técnico, apresentando uma terceira abordagem, escrita em Python, que se diferencia pela tipagem estática, validação automática de dados e geração nativa de documentação.

O fio condutor da aula é a construção progressiva de uma API para gerenciamento de veículos, contemplando: configuração do ambiente, criação de endpoints stateless (sem persistência), modelagem do recurso, implementação completa do CRUD (GET, POST, PUT, PATCH e DELETE) e, por fim, a evolução do projeto para persistir os dados em um banco de dados PostgreSQL utilizando o SQLAlchemy, ORM padrão de fato do ecossistema FastAPI.

2. Por que considerar o FastAPI?

O FastAPI é um framework web assíncrono para Python, construído sobre o Starlette (camada ASGI) e o Pydantic (validação e serialização de dados). Sua proposta central é usar as anotações de tipo (type hints) já nativas do Python para gerar, automaticamente, validação de entrada, serialização de saída e documentação interativa da API, sem a necessidade de escrever serializers ou arquivos de especificação separados.

ASGI (*Asynchronous Server Gateway Interface*) é uma especificação padrão do Python que conecta servidores web assíncronos a aplicações e frameworks web. Ele é o sucessor espiritual do WSGI (focado em requisições síncronas) e permite lidar com múltiplas conexões simultâneas, WebSockets e streaming de forma muito mais eficiente.

Aspecto	Django REST Framework	Node.js + Express	FastAPI
Paradigma	Orientado a classes, (ORM, admin, auth prontos)	Minimalista e não opinativo; tudo via middlewares e bibliotecas externas	Minimalista, porém com tipagem e validação nativas via type hints
Validação de dados	Serializers (DRF)	Manual ou com libs como Joi/Zod	Automática via Pydantic, baseada em type hints
Documentação da API	Manual ou com pacotes extras	Manual ou com Swagger configurado à parte	Gerada automaticamente (Swagger UI e ReDoc)
Desempenho	Síncrono por padrão (WSGI)	Assíncrono nativo (event loop)	Assíncrono nativo (ASGI)
Curva de aprendizado	Média/alta (muitas convenções)	Baixa para o básico, alta para arquitetura	Baixa a média; aproveita Python prévio

Três diferenciais serão explorados ao longo desta aula:

- Tipagem como contrato: os tipos declarados nos parâmetros das funções de rota definem automaticamente o que é aceito e o que é rejeitado pela API.
- Validação declarativa com Pydantic: modelos de dados (schemas) substituem validações manuais ou serializers extensos.
- Documentação automática: as interfaces Swagger UI (/docs) e ReDoc (/redoc) são geradas sem configuração adicional.

3. Configuração inicial do ambiente

Assim como em Django e Node, recomenda-se isolar as dependências do projeto. No ecossistema Python, isso é feito com ambientes virtuais (venv).

3.1 Criando o ambiente virtual

```
# Crie a pasta do projeto e acesse-a
mkdir api-veiculos-fastapi
cd api-veiculos-fastapi

# Crie o ambiente virtual
python3 -m venv venv

# Ative o ambiente virtual
# Linux/macOS:
source venv/bin/activate
# Windows (PowerShell):
venv\Scripts\Activate.ps1
```

3.2 Instalando as dependências

Com o ambiente ativo, instale o FastAPI e o Uvicorn (servidor ASGI responsável por executar a aplicação):

```
pip install fastapi uvicorn[standard]

# Congele as dependências do projeto (equivalente ao package.json do Node)
pip freeze > requirements.txt
```

Observação: O Uvicorn é um servidor web ASGI (Asynchronous Server Gateway Interface) ultrarrápido para Python

Observação: Vale comparar este passo com o `npm init / npm install` do Express e com o `pip install django` do Django, reforçando que, apesar das diferenças de sintaxe, o fluxo conceitual de gerenciamento de dependências é o mesmo nos três ecossistemas.

3.3 Estrutura inicial de pastas

```
api-veiculos-fastapi/
├── app/
│   ├── __init__.py
│   ├── main.py           # ponto de entrada da aplicação
│   ├── models.py         # schemas Pydantic (e, depois, modelos SQLAlchemy)
│   ├── database.py       # configuração de conexão com o banco (seção 8)
│   └── routers/
│       ├── __init__.py
│       └── veiculos.py   # rotas relacionadas ao recurso Veículo
├── requirements.txt
└── venv/
```

4. Primeiro endpoint: “Hello, API”

Crie o arquivo `app/main.py` com o código mínimo de uma aplicação FastAPI:

```
# app/main.py
from fastapi import FastAPI

app = FastAPI(
    title="API de Veículos",
    description="API construída na disciplina de PABD - TADS/IFRN",
    version="1.0.0",
)

@app.get("/")
def read_root():
    return {"mensagem": "API de Veículos no ar!"}
```

Para executar a aplicação, utilize o Fastapi dev apontando para o objeto `app` dentro do módulo `main`:

```
fastapi dev ou
python3 -m uvicorn app.main:app --reload
```

5. Modelando o recurso Veículo

No FastAPI, os “schemas” de entrada e saída de dados são definidos com o Pydantic, biblioteca de validação de dados baseada em type hints. Esses schemas cumprem, em parte, o papel que os Serializers cumprem no Django REST Framework.

Crie o arquivo `app/models.py` com o schema do veículo:

```
# app/models.py
from pydantic import BaseModel, Field
from typing import Optional

class VeiculoBase(BaseModel):
    placa: str = Field(..., min_length=7, max_length=8, examples=["ABC1D23"])
    marca: str = Field(..., min_length=2, max_length=50)
    modelo: str = Field(..., min_length=1, max_length=50)
    ano_fabricacao: int = Field(..., ge=1950, le=2026)
    cor: str = Field(..., max_length=30)
    quilometragem: float = Field(default=0.0, ge=0)

class VeiculoCreate(VeiculoBase):
    """Schema usado na criação (POST)."""
    pass

class VeiculoUpdate(BaseModel):
    """Schema usado na atualização parcial (PATCH) - todos os campos opcionais."""
    placa: Optional[str] = None
    marca: Optional[str] = None
    modelo: Optional[str] = None
    ano_fabricacao: Optional[int] = None
    cor: Optional[str] = None
    quilometragem: Optional[float] = None

class Veiculo(VeiculoBase):
    """Schema de resposta (inclui o identificador gerado pelo servidor)."""
```

```

id: int

class Config:
    from_attributes = True

```

Observe que a validação de obrigatoriedade, tipo e restrições (mínimo, máximo, intervalo numérico) é declarada diretamente nos atributos da classe, sem a necessidade de métodos `clean_*` (Django) ou bibliotecas externas de validação (Express).

6. CRUD stateless: GET, POST, PUT/PATCH e DELETE

Nesta etapa, os dados serão armazenados apenas em memória (uma lista Python), caracterizando endpoints stateless do ponto de vista de persistência: a cada reinicialização do servidor, os dados são perdidos. Essa abordagem é didaticamente útil para focar exclusivamente no roteamento, na validação e nos códigos de status HTTP, antes de introduzir a complexidade do banco de dados.

Verbo HTTP	Rota	Ação	Status esperado
GET	/veiculos	Lista todos os veículos cadastrados	200 OK
GET	/veiculos/{veiculo_id}	Retorna um veículo específico	200 OK / 404 Not Found
POST	/veiculos	Cria um novo veículo	201 Created
PUT	/veiculos/{veiculo_id}	Atualiza todos os campos de um veículo	200 OK / 404 Not Found
PATCH	/veiculos/{veiculo_id}	Atualiza parcialmente um veículo	200 OK / 404 Not Found
DELETE	/veiculos/{veiculo_id}	Remove um veículo	204 No Content / 404 Not Found

6.1 Criando o roteador de veículos

Crie o arquivo `app/routers/veiculos.py`:

```

# app/routers/veiculos.py
from fastapi import APIRouter, HTTPException, status
from app.models import Veiculo, VeiculoCreate, VeiculoUpdate

router = APIRouter(prefix="/veiculos", tags=["Veículos"])

# "Banco de dados" em memória (apenas para esta etapa stateless)
veiculos_db: list[dict] = []
proximo_id = 1

```

6.2 GET — listar e buscar por id

```

@router.get("/", response_model=list[Veiculo])
def listar_veiculos():
    return veiculos_db

@router.get("/{veiculo_id}", response_model=Veiculo)
def obter_veiculo(veiculo_id: int):

```

```

for veiculo in veiculos_db:
    if veiculo["id"] == veiculo_id:
        return veiculo
raise HTTPException(
    status_code=status.HTTP_404_NOT_FOUND,
    detail=f"Veículo com id {veiculo_id} não encontrado.",
)

```

O parâmetro `response_model` instrui o FastAPI a validar e serializar a saída de acordo com o schema informado, removendo automaticamente quaisquer campos extras presentes na estrutura interna.

6.3 POST — criar veículo

```

@router.post("/", response_model=Veiculo, status_code=status.HTTP_201_CREATED)
def criar_veiculo(dados: VeiculoCreate):
    global proximo_id
    novo_veiculo = dados.model_dump()
    novo_veiculo["id"] = proximo_id
    veiculos_db.append(novo_veiculo)
    proximo_id += 1
    return novo_veiculo

```

O parâmetro `dados: VeiculoCreate` é suficiente para que o FastAPI: (1) leia o corpo da requisição como JSON; (2) valide cada campo de acordo com o schema; e (3) rejeite a requisição com status 422 Unprocessable Entity caso algum campo esteja ausente ou em formato inválido — tudo isso sem código adicional.

6.4 PUT — atualização completa

```

@router.put("/{veiculo_id}", response_model=Veiculo)
def atualizar_veiculo(veiculo_id: int, dados: VeiculoCreate):
    for veiculo in veiculos_db:
        if veiculo["id"] == veiculo_id:
            atualizado = dados.model_dump()
            atualizado["id"] = veiculo_id
            veiculos_db[veiculos_db.index(veiculo)] = atualizado
            return atualizado
    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail=f"Veículo com id {veiculo_id} não encontrado.",
    )

```

6.5 PATCH — atualização parcial

```

@router.patch("/{veiculo_id}", response_model=Veiculo)
def atualizar_parcial_veiculo(veiculo_id: int, dados: VeiculoUpdate):
    for veiculo in veiculos_db:
        if veiculo["id"] == veiculo_id:
            campos_enviados = dados.model_dump(exclude_unset=True)
            veiculo.update(campos_enviados)
            return veiculo
    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail=f"Veículo com id {veiculo_id} não encontrado.",
    )

```

A diferença essencial entre PUT e PATCH está no uso de `exclude_unset=True`: apenas os campos efetivamente enviados pelo cliente são considerados, preservando os demais valores já existentes.

6.6 DELETE — remover veículo

```

@router.delete("/{veiculo_id}", status_code=status.HTTP_204_NO_CONTENT)

```

```
def remover_veiculo(veiculo_id: int):
    for veiculo in veiculos_db:
        if veiculo["id"] == veiculo_id:
            veiculos_db.remove(veiculo)
            return
    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail=f"Veículo com id {veiculo_id} não encontrado.",
    )
```

6.7 Registrando o roteador na aplicação

Atualize `app/main.py` para incluir o roteador de veículos:

```
# app/main.py
from fastapi import FastAPI
from app.routers import veiculos

app = FastAPI(
    title="API de Veículos",
    description="API construída na disciplina de PABD - TADS/IFRN",
    version="1.0.0",
)

app.include_router(veiculos.router)

@app.get("/")
def read_root():
    return {"mensagem": "API de Veículos no ar!"}
```

Observação para o professor: O uso de `APIRouter` para separar as rotas por recurso é conceitualmente equivalente aos routers do Express (`express.Router()`) e aos arquivos `urls.py` por app no Django, reforçando a ideia de modularização por domínio.

7. Documentação automática e testes manuais

Com o servidor em execução (`uvicorn app.main:app --reload`), duas interfaces de documentação ficam disponíveis automaticamente, geradas a partir da especificação OpenAPI:

- Swagger UI: <http://127.0.0.1:8000/docs> — interface interativa para testar cada endpoint diretamente no navegador.
- ReDoc: <http://127.0.0.1:8000/redoc> — documentação em formato de leitura, mais adequada para consulta e compartilhamento.

Recomenda-se, em sala, demonstrar o ciclo completo a partir do Swagger UI: criar um veículo com POST, listá-lo com GET, atualizá-lo com PATCH e removê-lo com DELETE, observando os códigos de status retornados a cada operação.

Alternativamente, os testes podem ser feitos via linha de comando com curl:

```
curl -X POST "http://127.0.0.1:8000/veiculos/" \
  -H "Content-Type: application/json" \
  -d '{"placa": "ABC1D23", "marca": "Fiat", "modelo": "Argo", "ano_fabricacao": 2022, "cor": "Branco"}'
```

8. Persistência com PostgreSQL e SQLAlchemy

O SQLAlchemy é o ORM de fato padrão utilizado em projetos FastAPI, da mesma forma que o ORM do Django é o padrão em projetos Django e o Prisma/Sequelize costuma ser adotado em projetos Node. Nesta seção, a API stateless construída anteriormente será conectada a um banco de dados PostgreSQL real.

8.1 Instalando as dependências de banco de dados

```
pip install sqlalchemy psycopg2-binary
pip freeze > requirements.txt
```

O pacote sqlalchemy fornece o ORM propriamente dito, enquanto o psycopg2-binary é o driver de conexão com o PostgreSQL.

8.2 Configurando a conexão (database.py)

```
# app/database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

# Ajuste usuário, senha, host, porta e nome do banco conforme seu ambiente
DATABASE_URL = "postgresql://usuario:senha@localhost:5432/veiculos_db"

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Observação para o professor: A função `get_db` utiliza o sistema de Dependency Injection do FastAPI (parâmetro `Depends`, usado na seção 9). Esse padrão é semelhante, em propósito, ao *middleware de conexão por requisição* usado em projetos Express com *pools de conexão*, e à *gestão de conexões feita implicitamente pelo Django ORM*.

8.3 Definindo o modelo ORM

Adicione ao `app/models.py` o modelo SQLAlchemy correspondente à tabela de veículos. Para clareza didática, ele é mantido no mesmo arquivo dos schemas Pydantic, mas em projetos maiores costuma-se separá-los (ex.: `orm_models.py`):

```
# app/models.py (acrescentar ao arquivo existente)
from sqlalchemy import Column, Integer, String, Float
from app.database import Base

class VeiculoORM(Base):
    __tablename__ = "veiculos"

    id = Column(Integer, primary_key=True, index=True)
    placa = Column(String(8), unique=True, nullable=False, index=True)
    marca = Column(String(50), nullable=False)
    modelo = Column(String(50), nullable=False)
    ano_fabricacao = Column(Integer, nullable=False)
    cor = Column(String(30), nullable=False)
    quilometragem = Column(Float, default=0.0, nullable=False)
```

8.4 Criando as tabelas no banco

Para fins didáticos, as tabelas podem ser criadas diretamente a partir dos modelos declarados, adicionando ao final de `app/main.py`:

```
# app/main.py (acrescentar)
from app.database import Base, engine
from app import models

Base.metadata.create_all(bind=engine)
```

Observação para o professor: Em projetos profissionais, recomenda-se substituir `create_all` por um sistema de migrações (Alembic), de forma equivalente ao `makemigrations/migrate` do Django. Para o nível introdutório da disciplina, `create_all` é suficiente.

9. Refatorando o CRUD para usar o banco de dados

Com a conexão e o modelo ORM definidos, o roteador de veículos é reescrito para persistir os dados em PostgreSQL por meio de uma `Session` do SQLAlchemy, injetada automaticamente em cada rota via `Depends(get_db)`.

```
# app/routers/veiculos.py (versão com persistência)
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from app.database import get_db
from app.models import Veiculo, VeiculoCreate, VeiculoUpdate, VeiculoORM

router = APIRouter(prefix="/veiculos", tags=["Veículos"])

@router.get("/", response_model=list[Veiculo])
def listar_veiculos(db: Session = Depends(get_db)):
    return db.query(VeiculoORM).all()

@router.get("/{veiculo_id}", response_model=Veiculo)
def obter_veiculo(veiculo_id: int, db: Session = Depends(get_db)):
    veiculo = db.query(VeiculoORM).filter(VeiculoORM.id == veiculo_id).first()
    if not veiculo:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Veículo não encontrado.")
    return veiculo

@router.post("/", response_model=Veiculo, status_code=status.HTTP_201_CREATED)
def criar_veiculo(dados: VeiculoCreate, db: Session = Depends(get_db)):
    novo_veiculo = VeiculoORM(**dados.model_dump())
    db.add(novo_veiculo)
    db.commit()
    db.refresh(novo_veiculo)
    return novo_veiculo

@router.put("/{veiculo_id}", response_model=Veiculo)
def atualizar_veiculo(veiculo_id: int, dados: VeiculoUpdate, db: Session = Depends(get_db)):
    veiculo = db.query(VeiculoORM).filter(VeiculoORM.id == veiculo_id).first()
    if not veiculo:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Veículo não encontrado.")
    for campo, valor in dados.model_dump().items():
        setattr(veiculo, campo, valor)
    db.commit()
    db.refresh(veiculo)
    return veiculo
```

```

@router.patch("/{veiculo_id}", response_model=Veiculo)
def atualizar_parcial_veiculo(veiculo_id: int, dados: VeiculoUpdate, db: Session = Depends(get_db)):
    veiculo = db.query(VeiculoORM).filter(VeiculoORM.id == veiculo_id).first()
    if not veiculo:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Veículo não encontrado.")
    for campo, valor in dados.model_dump(exclude_unset=True).items():
        setattr(veiculo, campo, valor)
    db.commit()
    db.refresh(veiculo)
    return veiculo

@router.delete("/{veiculo_id}", status_code=status.HTTP_204_NO_CONTENT)
def remover_veiculo(veiculo_id: int, db: Session = Depends(get_db)):
    veiculo = db.query(VeiculoORM).filter(VeiculoORM.id == veiculo_id).first()
    if not veiculo:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Veículo não encontrado.")
    db.delete(veiculo)
    db.commit()

```

Vale destacar, que a assinatura das rotas mudou muito pouco em relação à versão stateless: apenas o parâmetro `db: Session = Depends(get_db)` foi adicionado, e as operações sobre a lista em memória foram substituídas por chamadas equivalentes do SQLAlchemy (`query`, `add`, `commit`, `delete`). Essa transição suave é um dos pontos pedagógicos mais relevantes da aula.

9.1 Comparando com os ORMs já estudados

Operação	Django ORM	SQLAlchemy (FastAPI)
Listar todos	<code>Veiculo.objects.all()</code>	<code>db.query(VeiculoORM).all()</code>
Buscar por id	<code>Veiculo.objects.get(id=x)</code>	<code>db.query(VeiculoORM).filter(VeiculoORM.id == x).first()</code>
Criar	<code>Veiculo.objects.create(**dados)</code>	<code>db.add(VeiculoORM(**dados)); db.commit()</code>
Atualizar	<code>objeto.save()</code>	<code>setattr(...); db.commit()</code>
Remover	<code>objeto.delete()</code>	<code>db.delete(objeto); db.commit()</code>

10. Exercícios propostos e próximos passos

1. Adicionar um novo campo `proprietario_cpf` ao modelo de veículo, incluindo validação de formato (regex) no schema Pydantic.
2. Implementar paginação na rota `GET /veiculos` usando os parâmetros de query `skip` e `limit`.
3. Implementar um filtro por marca na rota de listagem, utilizando um parâmetro de query opcional.
4. Adicionar tratamento de erro para placas duplicadas, retornando 409 Conflict ao tentar cadastrar uma placa já existente.
5. Comparar, em um pequeno relatório, o tempo de desenvolvimento e a quantidade de código necessária para implementar o mesmo CRUD de veículos em Django REST Framework, Express e FastAPI.

Como aprofundamento, sugere-se explorar nas próximas aulas: autenticação com OAuth2 e JWT no FastAPI (`security.OAuth2PasswordBearer`), endpoints assíncronos (`async def`) combinados com drivers assíncronos do PostgreSQL (`asyncpg`) e o uso do Alembic para versionamento de migrações de banco de dados.